Airline Passengers: Satisfaction Recognition through Machine Learning

Barera Caterina Flavia (mat.852060) - Malvezzi Beatrice (mat.907500) - Renato Martina (mat.842598)

Introduction

Abstract

In the implementation of clustering methods, including DBSCAN, K-means & hierarchical clustering, we've obtained satisfying results. Especially in the case of hierarchical clustering, it was able to separate the satisfied and the unsatisfied customers reasonably well. DBSCAN has been particularly useful for the detection of the most influential extreme values. Subsequently, in the classification part, we have trained several algorithms named Random Forest, KNN-9, Adaboost and lastly Support Vector Machine with a linear kernel. We've then evaluated their performance. The Random Forest algorithm turned out to be the best performer and led to an accuracy equal to 94%.

The dataset used for this analysis can be found at the following link "https://www.kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction".
It comprehends 25,976 instances and 25 variables. Each observation corresponds to an airline passenger who was asked to evaluate their travel experience after the flight.

Goals

The main purpose of the following analysis was to investigate whether the survey conducted by the airline company is helpful to understand the degree of satisfaction of a potential customer or if it can be improved in some ways.

- Firstly, our aim was to implement some clustering methods using only the rating features, to understand if they are able to create homogeneous groups that match with our two levels, without knowing the labels of our target variable. Furthermore, we were interested in exploring if it's possible to create other kinds of interesting segmentation of the respondents through these clustering methods.

- Secondly, some classification methods were used with the purpose to find the best one in terms of accuracy, ability to generalise and other indicators. It's also important to evaluate which features have a bigger impact in the classification; this last point may be important in assisting the airline company to omit certain questions from the survey in order to reduce the load for the respondents and lower the risk of losing interest during the compilation process.

Description of the dataset

As seen in the previous section, the dataset contains the results of a customer satisfaction study submitted to the passengers of an airline company.

The 25 variables used in our analysis examine different aspects of the customers experience and they can be categorised as:

1. FACTS: demographic variables which describe the characteristics of the customers and objective information about the type of the fight:
   - `x`: ordered sequence from 1 to the number of responders;
   - `id`: responder identification number;
   - `Age`: the actual age of the passengers;
   - `Gender`: the gender of the passengers (Female, Male);
   - `Customer Type`: variable that shows the customer type (Loyal, Disloyal);
   - `Type of Travel`: the passengers' purpose of the flight (Personal Travel, Business Travel);
   - `Class`: the travel class in the plane of the passengers (Business, Eco, Eco Plus);
   - `Flight distance`: the flight distance of the passengers' journey;
   - `Departure Delay in Minutes`: minutes delayed when departing;
   - `Arrival Delay in Minutes`: minutes delayed when arriving.
2. OPINIONS: subjective and personal judgements regarding different aspects of the service. The variables dealing with these assessments require a vote between 1 and 5 and examine the following perspectives:
   - `Inflight wifi service`: satisfaction level of the inflight wi-fi service;
   - `Departure/Arrival time convenient`: satisfaction level of departure and arrival time convenient;
   - `Ease of Online booking`: satisfaction level of online booking;
   - `Gate location`: satisfaction level of gate location;
   - `Food and drink`: satisfaction level of food and drink;
   - `Online boarding`: satisfaction level of online boarding;
   - `Seat comfort`: satisfaction level of seat comfort;
   - `Inflight entertainment`: satisfaction level of inflight entertainment;
   - `On-board service`: satisfaction level of on-board service;
   - `Leg room service`: satisfaction level of leg room service;
   - `Baggage handling`: satisfaction level of baggage handling;
   - `Check-in service`: satisfaction level of check-in service;
   - `Inflight service`: satisfaction level of inflight service;
   - `Cleanliness`: satisfaction level of cleanliness.

Besides the explanatory variables mentioned earlier, the key variable of interest is "`Satisfaction`". It indicates whether the passenger is happy ("*satisfied*") or not ("*neutral or dissatisfied*") regarding the overall flight experience.

Pre-processing

Feature analysis

The target feature of our analysis is named `Satisfaction`. It's a dichotomous variable and it has two different levels which are "*satisfied*" and "*neutral or dissatisfied*". This feature allows us to identify the customers who are generally satisfied with the service from those who are not. As shown in Figure 1, there's a slight imbalance between the two classes. Since the majority of the customers fall into the "*neutral or dissatisfied*" class, one possible explanation for this result is that this class may be seen as a group including two different kinds of clients: the neutral and the dissatisfied. Furthermore, in the pre-processing step the target variable is analysed according to the factor explanatory variables, and it has been found that the travel class (`Class`) has the strongest influence among all variables. The conditional distribution is depicted below (Figure 2).

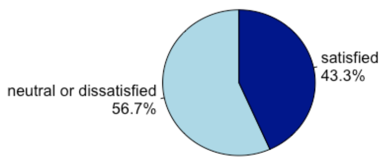

Figure 1: satisfaction pie chart

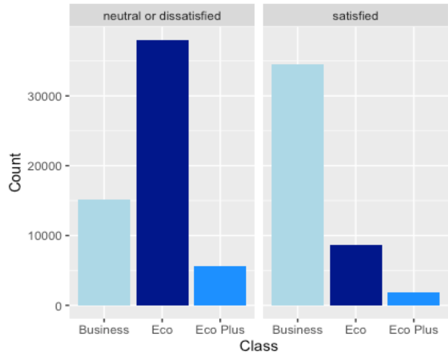

Figure 2: conditional distribution for "class"

Regarding the explanatory variables, the attributes located in the first two positions have been deleted as they were irrelevant for our study as they uniquely identify the dataset rows. For the remaining variables, we assessed techniques such as standardisation, as we planned to use distance-based clustering methods. Boxplot graphs are presented below to provide an indication of the presence of outliers in `Departure.Delay` and in `Arrival.Delay`, which we later addressed in our analysis using the DBSCAN algorithm.

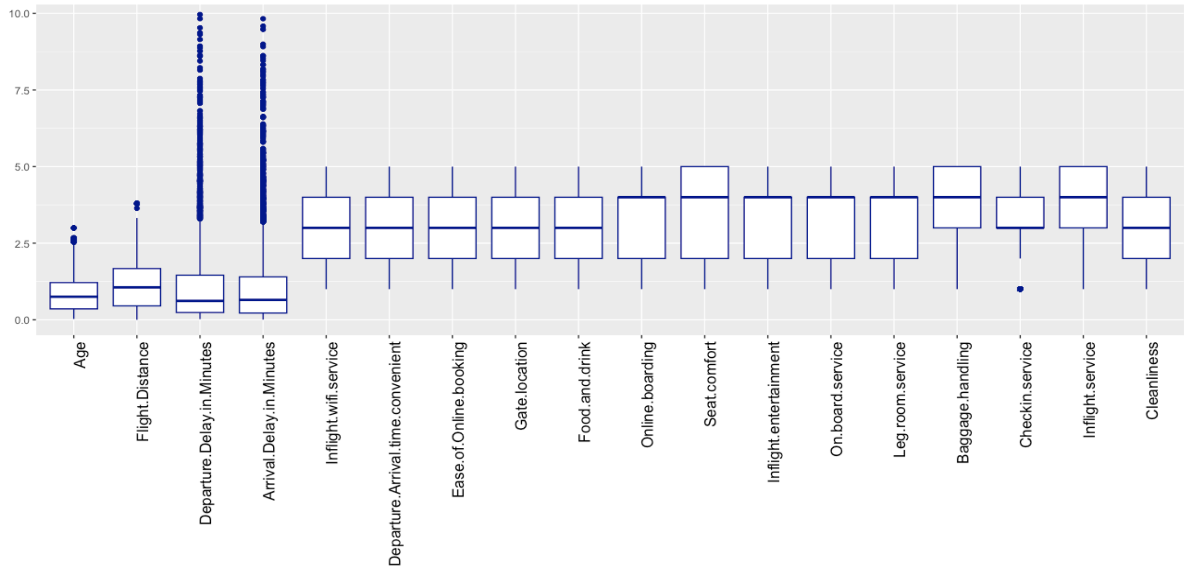

Figure 3: explanatory variables boxplot

Missing values analysis

The first step of our pre-processing analysis consisted of looking for the presence of missing values. Upon analysing the evaluation features, it's been possible to notice the presence of votes equal to zero, that we treated as missing values. Indeed, according to the exogenous information available in the dataset source, the evaluation scale is specified to be between 1 and 5. Beyond that, as it can be seen from Figure 4, the majority of the NAs is located in the feature "`Arrival Delay in Minutes`". Although the percentages show a limited presence of NAs, it was deemed appropriate to give them an adequate treatment in order to avoid losing additional information. It has been decided to proceed with an imputation of medians for the rating features and with an imputation with the MICE (Multivariate Imputation by Chained Equations) R package for the remaining ones.

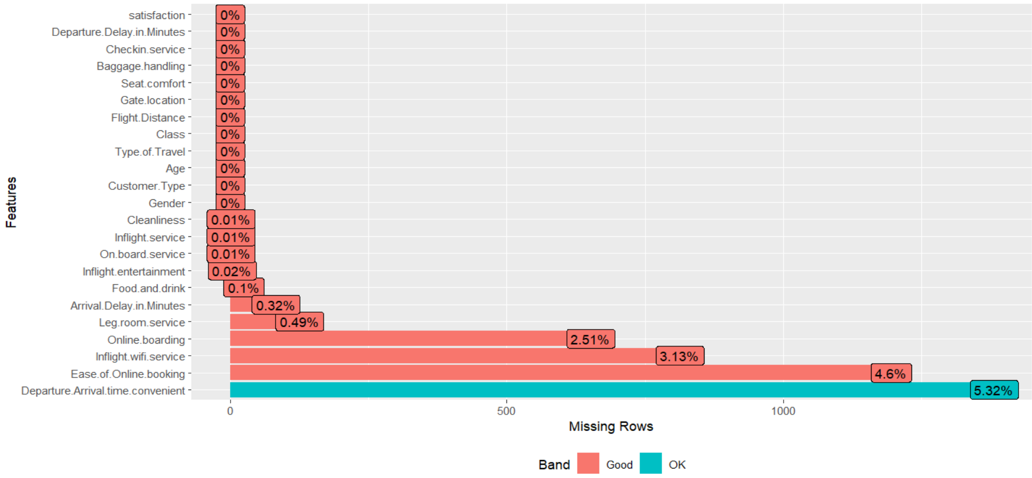

Figure 4: percentage of NA values in the training set

Outlier Analysis

The analysis proceeded by analysing the outliers through graphs such as boxplot (Figure 5). The most irregular features are `Departure Delay in Minutes` and `Arrival Delay in Minutes`. One possible explanation is that their distribution is heavily concentrated around zero, indicating that the biggest majority of flights for this airline company is on time. Most outliers (≈95%) for these two features are related to the same instances in the dataset, suggesting a strong connection between them (this will be further confirmed with the correlation analysis). This finding is consistent with the usual occurrence: when a flight departs late then it will probably land late. To gain a better understanding on how to treat these outliers a DBSCAN algorithm was implemented later on.

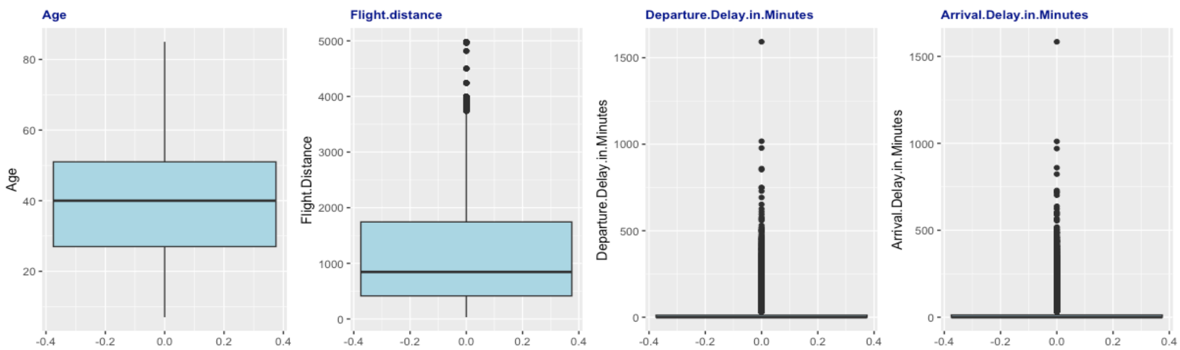

Figure 5: focus on boxplots which show outlier observations.

Correlation analysis

The goal of the explorative analysis is to characterise the type of relationship existing between the response variable and the explanatory variables, as well as among the explanatory variables themselves. First of all, we studied the correlation between the quantitative variables. Subsequently to the initial analysis, we concluded a correlation analysis among the numerical features of the dataset. As we can see in the corrplot in Figure 6, the only significant correlation is the one between `Departure Delay in Minutes` and `Arrival Delay in Minutes`, which confirms the hypothesis stated during the outlier analysis. For this reason, it has been decided to not consider the `Arrival Delay in Minutes` feature in the following analysis to avoid multicollinearity problems which can affect the results.

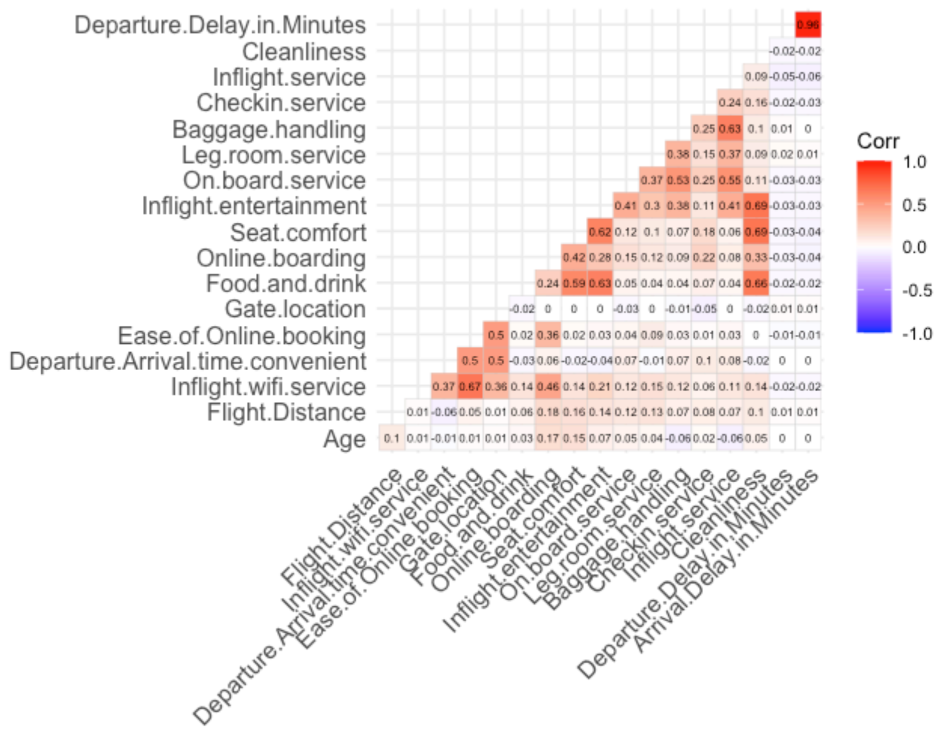

Figure 6: Corrplot between numerical features

Unsupervised learning: clustering

In this section we applied unsupervised learning algorithms with the aim of determining if it is possible to divide respondents into two groups based on their satisfaction level, without considering the `satisfaction` target variable, but only by using the evaluation variables.

Dbscan for the outlier analysis

The Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm is effective in handling noisy points and it doesn't require previous knowledge of the number of clusters so it can be used effectively for anomaly detection, as it has been done in this study.

In order to assess the values of ε and the minimum threshold of data points for clustering, the algorithm was iterated and tested based on its performance in terms of silhouette. The results showed that the algorithm achieved the best performance when ε was set on 3.5, and the minimum threshold of data points was set to 18.

```
Best ε: 3.5
Best minimum threshold τ: 18
Time taken: 33.19 sec elapsed
```

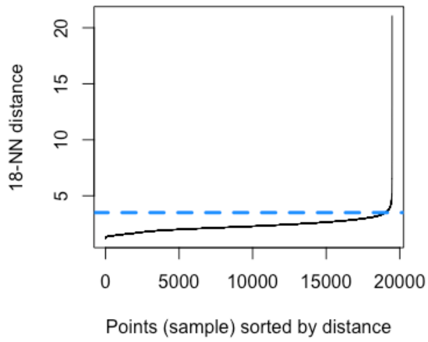

Figure 7: KNN graph

After investigating the results, we noticed that there's a correspondence between the most extreme values, especially of the `Departure Delay in minutes` feature and the observations DBSCAN put inside the noisy points' group. As a result, it has been decided to drop these observations as this algorithm has confirmed their strong impact. This choice is also justified by the fact that, within the group of extreme observations, the percentage of "*satisfied*" and "*neutral or dissatisfied*" doesn't change. This suggests that these instances do not significantly contribute to the discrimination of our target feature.

Principal Component Analysis

Since we observed that some of the rating variables are correlated (Figure 6), we agreed to apply the Principal Component Analysis (PCA) to reduce the dimensionality of the original dataset, while losing the least amount of information possible. This approach aims to make the algorithm computationally less intensive.

Before proceeding with PCA, we perform some tests to verify the suitability of this analysis.

- *Bartlett's test of sphericity*, which tests whether a correlation matrix is significantly different from an identity matrix or not. Since the test follows a Chi-square distribution and the p-value is significant at an alpha level of .05, the correlation matrix is suitable for factor analysis because the variables show covariance.
- *KMO test*, which studies the sampling adequacy using the partial correlation coefficients between variables. We found a KMO value high enough (0.79), which represents the degree to which each observed variable is predicted by the other variables in the dataset and this indicates the suitability for factor analysis.

Once all the conditions were identified, it was possible to proceed with the analysis. Based on the scree-plot graph (Figure 8), the number of components to use is determined by looking at the elbow point: in this particular case 7 components were used. This choice is supported by the fact that the percentage of cumulative variability explained by the first 7 components is sufficiently high. Also, the Kaiser criterion is satisfied as we use only the components with eigenvalues higher than 1.

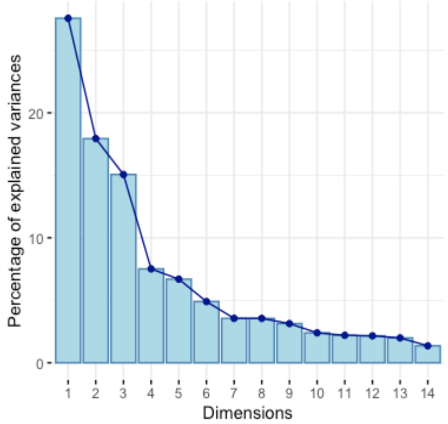

Figure 8: Scree plot graph for PCA

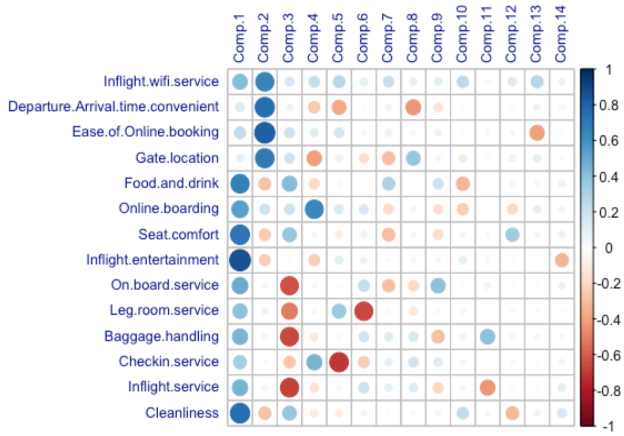

Figure 9: Representation of the components' matrix

The components used in the following clustering algorithms can be interpreted on the basis of the components' matrix (Figure 9). In the table below, the naming of the components is reported based on the variables that are most strongly correlated with each component.

Component	Name	variables
• Comp1:	*"Inflight experience"*	- `food and drink,` - `seat comfort,` - `inflight entertainment,` - `cleanliness`
• Comp2:	*"Digital convenience"*	- `inflight wifi service,` - `ease online booking`
• Comp3:	*"Service quality"*	- `on.board.service,` - `baggage.handling` - `inflight.service`
• Comp4:	*"Online boarding"*	referred to the homonymous variable
• Comp5:	*"Check-in service"*	referred to the homonymous variable
• Comp6:	*"Leg room comfort"*	referred to the homonymous variable
• Comp7:	*"Timeliness and accessibility"*	- `departure.arrival.time.convenience,` - `gate.location`

Iterative- based distance: K-means

PCA's components are used in the k-means algorithm to divide the data into a predefined number of K distinct clusters with the Euclidean distance.

There are different ways to choose the adequate number of groups and they can lead to different results. The first one we used is based on the ratio among within and total distance, which naturally decreases alongside the increasing number of clusters. Looking at the graph (Figure 10), there's a change in the slope in correspondence of $k = 4$ (`1.38 sec elapsed`), which appears to be the best, based on this technique. Despite this, upon exploring the four clusters obtained, we found no significant differences between them. For this reason, an alternative approach based on *silhouette analysis* (Figure 11) has been adopted to try to find the best number of clusters, resulting in $k = 2$ (`0.47 sec elapsed`).

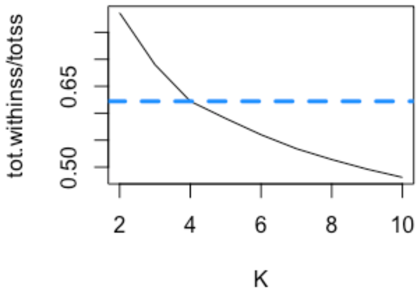

Figure 10: K maximisation through tot.withinss/totss

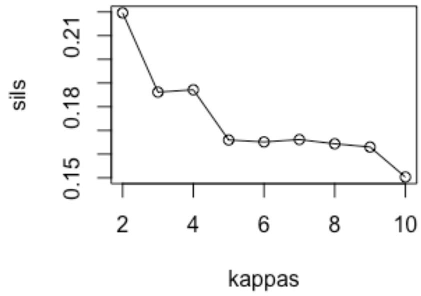

Figure 11: K maximisation through silhouette

In this case, the algorithm is able to split the observations especially relying on the first principal component, which refers to the flight experience. The two clusters show the same proportion of "*satisfied*" and "*neutral or dissatisfied*" customers as the original dataset, meaning that the k-means hasn't been able to identify the two levels of our target feature. Nevertheless, the rating features' mean seem slightly lower for one of the two clusters created. However, we believed that this kind of approach is overly simplistic for the current scenario, so we opted to apply hierarchical clustering, in order to achieve a better splitting.

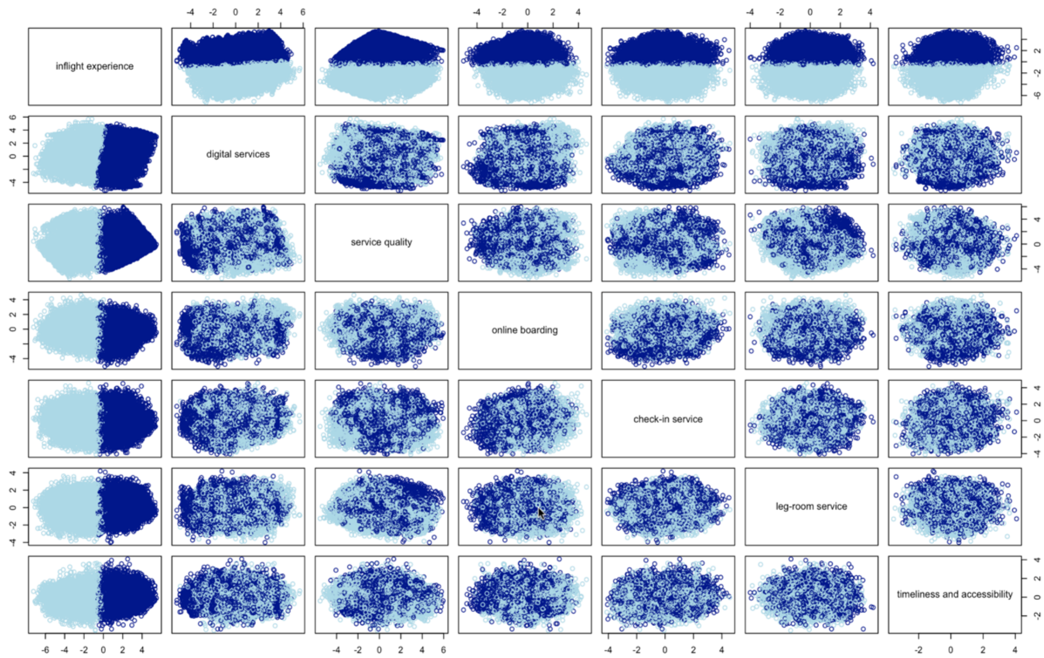

Figure 12: conditional scatterplot of the clustering with $k = 2$

Hierarchical clustering

The second method we used is the agglomerative hierarchical clustering. In this case, we chose to use the data synthesised through PCA.

To calculate the distance matrix, we tried both Euclidean and Manhattan distances. For both distances the best results are achieved by selecting the Ward linkage aggregation method, which merges clusters with the least increase in the sum of squared distances after the linkage. In this way, the Ward method creates groups that are more dissimilar to each other and more similar within themselves.

From the dendrograms (Figure 13 & 14) we can see that the partitioning is reasonably good, because the group structures are differentiated. The number of groups k has been obtained through the optimization of the silhouette score and the dendrograms are cut into two groups using Manhattan and Euclidean distance measures.

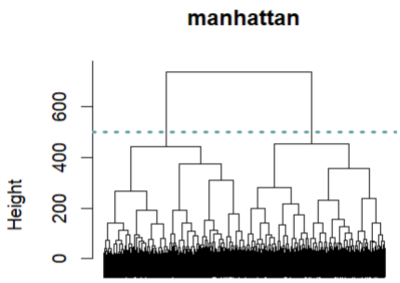

Figure 13: dendrogram

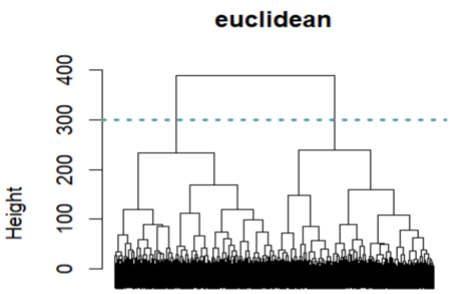

Figure 14: dendrogram

In both results the clustering is reasonably able to divide "*satisfied*" passengers from "*neutral or dissatisfied*", especially using the euclidean distance.

Euclidean distance works very well with variables that have the same unit of measurement and it shows the best results in terms of silhouette and proportion for our target feature, so we decided to keep this distance measure.

```
Best distance: euclidean
Best k: 2
Time taken: 17.72 sec elapsed
```

It leads to two clusters where one has almost 74.9% of "*neutral or dissatisfied*" clients and the other one 65.2% "*satisfied*". Moreover, this is confirmed by looking at the rating average since there's a significant difference between the two groups. As it can be seen from Figure 16, the average of each one of the evaluation features tends to be higher in the first group than in the second one, especially for the features "`cleanliness`", "`food and drink`", "`seat.comfort`" and "`inflight.entertainment`", which compose the "`flight experience`" component.

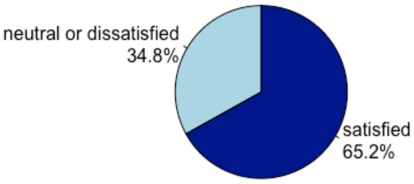

Figure 15: Pie chart for cluster 1

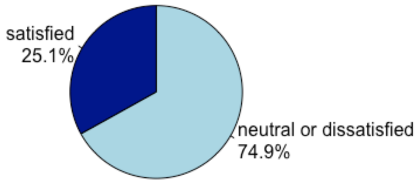

Figure 16: Pie chart for cluster 2

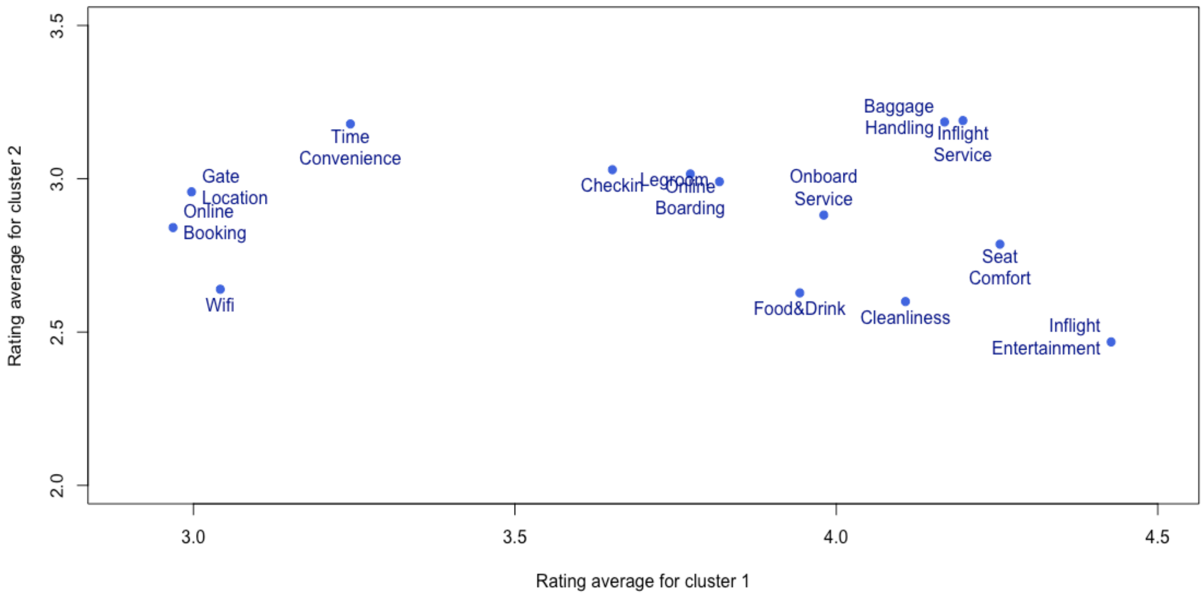

Figure 17: Scatterplot to compare the clusters (rating variables means)

Supervised Learning: Classification

Before proceeding with further analysis, the dataset has been split into train (75%) and test (25%).

Random forest

The first classification algorithm we implemented to the dataset is the Random Forest. The hyperparameter that needs to be optimised is the number of trees. The general Out-Of- Bag error (red line) drops fast for a low number of trees and then it becomes steady by trees=80 (Figure 18). The same happens for the OOB classification error referring to single levels. It means that 80 trees are enough to minimise the global error. After this consideration we decided to implement the algorithm using this hyperparameter starting from the training set, which led us to this following outcome:

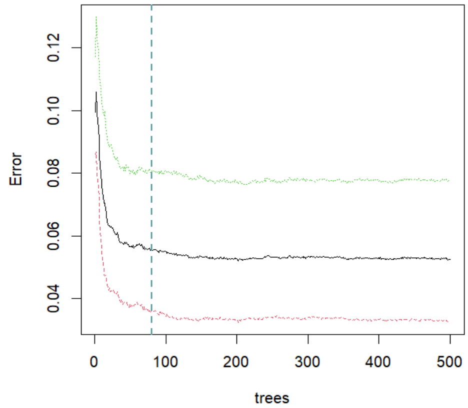

Figure 18: Relation between errors and number of trees

```
OOB estimate of error rate: 5.51%
Classification accuracy: 94.49%
Sensitivity: 96.41%
Specificity: 92.06%
Time taken: 8.42 sec elapsed
```

Confusion matrix

		actual 0	actual 1
fitted	0	10458	679
	1	389	7874

In light of these results, the accuracy obtained in the training set seems good. Taking a closer look, the specificity and sensitivity are pretty high, especially the latter. A reason for this is that the specificity refers to the satisfied customers (actual=1) which had shown a slight imbalance during the initial analysis compared to the other level. So, the algorithm in doubtful situations tends to classify as the majority level.

This algorithm is usually used also to check the importance that each feature has in the classification (Figure 19), both in terms of the mean decrease in Gini coefficient and the mean decrease in accuracy. The higher these indicators are for a feature, the bigger its impacts in the classification.

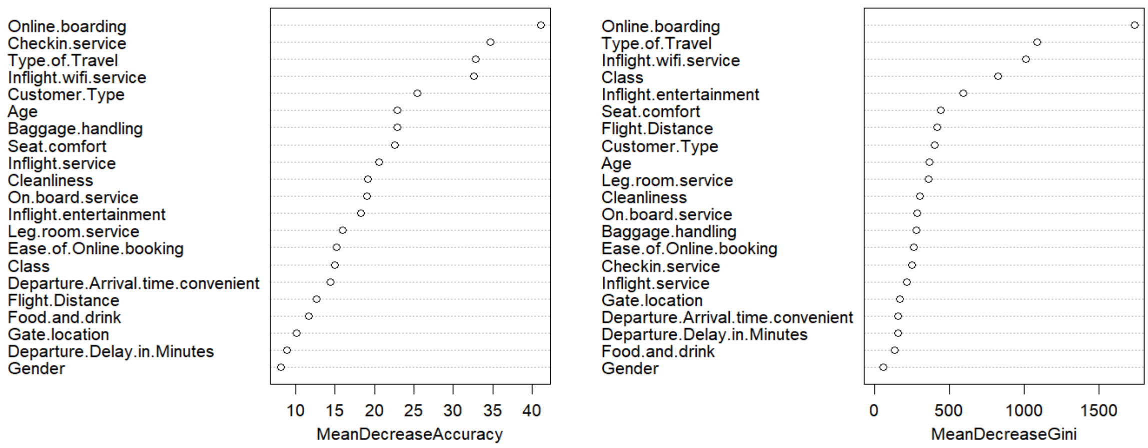

Figure 19: Variable importance plot for random forest with 80 trees

In our case the most important feature in both variants is the `Online.boarding`, while the most irrelevant one refers to `Gender`. As suspected during the outlier analysis, `Departure.Delay.in.Minutes` doesn't really help to explain the satisfaction as a customer. The feature `Class`, which had already shown a good capability in discriminating the levels graphically (Figure 2), is one of the most important in terms of Gini but it loses positions in terms of accuracy of the method. The less impactful rating features in both cases are `Food.and.drink` and `Gate.location` which may be not helpful to understand the level of satisfaction of a customer and then subsequently dropped from the survey if the aim is to maintain high the attention of the respondents. Subsequently, the random forest algorithm was applied to the test set to check the generalisation ability of our ensemble method:

```
Test accuracy: 94.78%
Sensitivity: 96.36%
Specificity: 92.71%
```

Confusion matrix

		actual 0	actual 1
fitted	0	3546	205
	1	134	2609

The test accuracy remains stable which means that the algorithm has a good performance even with never-used instances. To check deeply its behaviour, it's important to check also the sensitivity and specificity values; the last one shows lower values, coherently to what happened in the training set.

K-Nearest Neighbour: KNN-9

K-Nearest Neighbour (KNN) is a simple but powerful, instance-based algorithm. It operates based on similarity of data points. The aim of KNN is to maximise the accuracy to classify at best the target variable. The hyperparameter that needs to be optimised is the number of neighbours through an iteration which tries to find the k which minimises the test error rate. The minimum point is reached by $k = 10$ (Figure 20), but in a binary context it is preferable to use an odd number of neighbours to avoid doubtful situations. For this reason, the value chosen for the KNN algorithm is 9, which corresponds to the second lowest test-error rate. In conclusion, the hyperparameter optimization led to choosing a KNN with $k = 9$ nearest neighbours using the metric of Euclidean distance. The results of our algorithm are shown below:

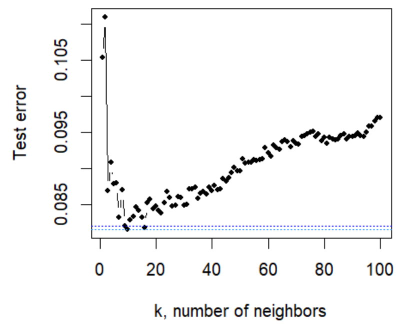

Figure 20: Test error rate VS number of neighbours

```
Test accuracy: 91.81 %
Sensitivity: 87.81%
Specificity: 94.86%
Time taken: 4.84 sec elapsed
```

Confusion matrix

		actual 0	actual 1
fitted	0	3491	343
	1	189	2471

AdaBoost with tree classifiers

AdaBoost, short for Adaptive Boosting, is a sequential ensemble method in which each component is built based on the errors made by the previous one. We decided to apply the AdaBoost algorithm to decision tree classifiers. The basic version requires a dichotomous variable used as target feature, with two levels, which should be labelled as -1 and +1. The hyperparameters that need to be chosen are the depth of the base tree classifier and the number of boosting rounds to use.

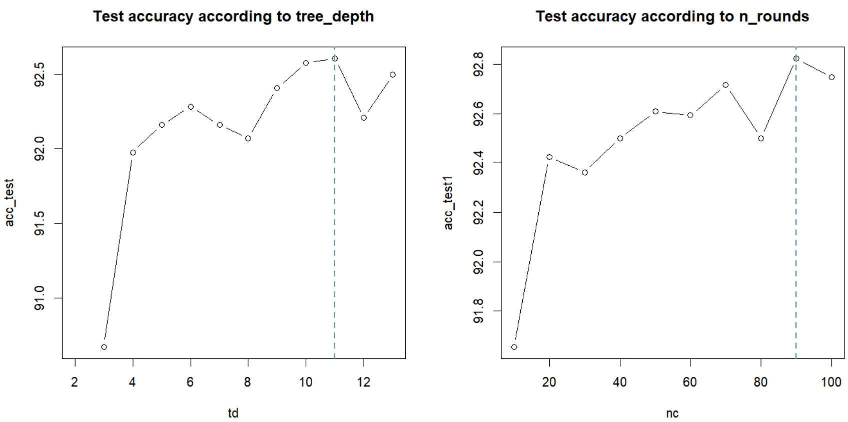

Figure 21: Hyperparameters choice for Adaboost algorithm

As a result of our study, the optimal combination that maximises the test accuracy is tree_depth=11 and n_rounds=90. The results below are obtained in training set:

```
Training accuracy: 100%
Sensitivity: 100%
Specificity: 100%
Time taken: 46.44 sec elapsed
```

Confusion matrix

		actual 0	actual 1
fitted	0	10847	0
	1	0	8553

All the indicators are very high, reaching perfection. However, this is not always a positive sign because it could mean that the method fits too well to the training set and struggles to generalise well to the test set. To evaluate its generalisation ability, it is crucial to assess the performance achieved in the test set:

```
Test accuracy: 92.82%
Sensitivity: 90.94%
Specificity: 94.27%
```

Confusion matrix

		actual -1	actual 1
fitted	-1	3469	255
	1	211	2559

The results show that in the test set the accuracy gets worse as well as the other indicators, particularly the sensitivity. This implies that the algorithm tends to fit really well to the initial data, which poses the risk of weakening its performance when presented with new observations.

Support Vector Machine (SVM) with a linear Kernel

Support Vector Machine (SVM)'s aim is to find an optimal decision boundary by maximising the margin and correctly classifying the data points. Since a linear kernel is used, the only hyperparameter that needs to be optimised is the cost; the goal is to find the hyperparameter C that minimises the test-error; this happens for a value equal to 0.5. For this reason, a SVM algorithm with this hyperparameter has been implemented in the training set, which has led to the following results:

```
Training accuracy: 88.64%
Sensitivity: 86.69%
Specificity: 90.17%
Time taken: 32.3 sec elapsed
```

Confusion matrix

		actual 0	actual 1
fitted	0	9781	1138
	1	1066	7415

Then, it has been used to predict unseen data of the test set:

```
Test accuracy: 89.16%
Sensitivity: 87.17
Specificity: 90.68%
```

Confusion matrix

		actual 0	actual 1
fitted	0	3337	361
	1	343	2453

Both test and training accuracy are lower than the ones obtained in the previous classification algorithms. Despite this, its performance improves moving from the training to the test, which means that it doesn't overfit the initial data and it has a good ability to generalise, but just in the limited context of this algorithm.

Conclusions

In addition to the indicators already computed, another way to compare and to select the best classification algorithm is by calculating the Receiver Operating Characteristic (ROC) curve and the related Area Under the Curve (AUC), which show the general performance of each method. Based on these results, the Random Forest demonstrated the best performance and also yielded the best test accuracy. As noted earlier, it is also the second fastest model, which is advantageous as it means that the Random Forest is also less computationally intensive than other methods. On the other hand, the worst performance in terms of AUC and other indicators was demonstrated by the Support vector machine with linear kernel.

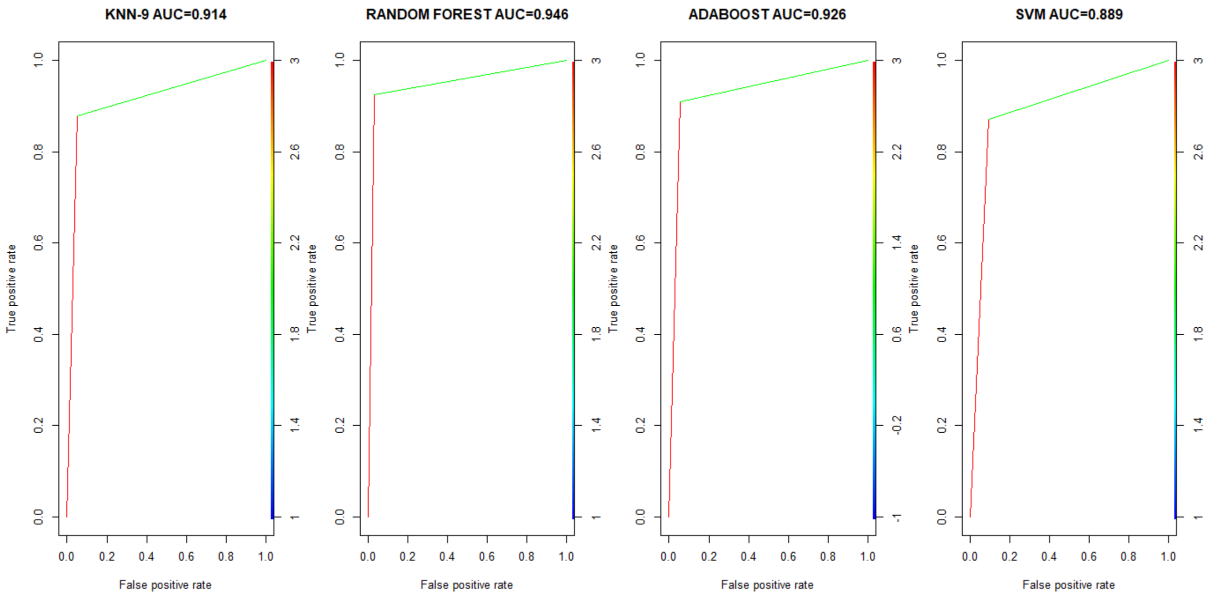

Figure 22: ROC Curve for the algorithm used in our analysis.

In conclusion, the Random Forest algorithm is considered the best classification algorithm, even though all algorithms performed well in both the training and the test set, despite the imbalance of the target feature. This analysis shows non-trivial conclusions about the most influential variables for the response, which indicates that the dissatisfaction of the customers depends more on some service like `Online.boarding`, and less on some other ones, like `Food.and.drink`. These findings are important for airlines as satisfied customers would be more likely to repurchase tickets from the same airline company; for this reason, they need to identify which areas they need to invest in, to improve their facilities and make their clients more satisfied.

References

[1] Dataset available at https://www.kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction
[2] https://elearning.unimib.it/course/view.php?id=44929
[3] James G., Witten D., Hastie T., Tibshirani R. An Introduction to Statistical Learning
[4] Statistica per Data Science con R - http://www.r-project.it/_book/